Plan semanal hasta el 27/11 (70%)

24 oct 2026 · @Someone

Resumen

Con **2 ítems por integrante por semana** durante 4 semanas se cubre el core: 33 de las 46 US del Trello (72%) más las tareas T-03, T-05 y T-06. La semana del 21/11 queda libre para integrar, probar el camino feliz y cerrar la documentación.

- Semanas de trabajo: S1 a S4, del sábado 24/10 al viernes 20/11. Cada sábado hay reunión de planificación.
- S5 (21/11 al 27/11) es de congelamiento: no entran US nuevas, solo bugs, casos de prueba faltantes y documentos.
- Una US cuenta como hecha solo con su código integrado y sus casos de prueba escritos en la planilla, en la misma semana.
- Si alguien termina antes, toma una US secundaria de su propio bloque. Así se puede pasar del 70% sin desordenar el plan.

Plan ajustado al tablero de Trello del 24/10. Lo que encontré inconsistente o sin cargar en el tablero está al final, en «Ajustes al Trello».

Qué entra en el 70% y qué queda para el 11/12

El core es todo lo que hace falta para recorrer el camino feliz de punta a punta con los 5 roles. El camino feliz es: el Administrador activa su cuenta, crea un barrio, crea y asigna un Operador; el Operador crea un Guardia; una Empresa se registra y el Administrador la aprueba; el Operador la habilita; la Empresa carga un trabajador con documentación vigente; el Guardia lo busca por DNI, verifica y registra ingreso y egreso; el Operador ve el historial.

Épica	Core para el 27/11 (33 US)	Secundario para el 11/12 (13 US)	
E01 Autenticación y cuentas	US-01 Activar cuenta del Admin, US-02 Crear cuenta, US-03 Solicitar perfil, US-04 Iniciar sesión, US-05 Cerrar sesión, US-08 Ver estado de mi solicitud	US-06 Recuperar contraseña (depende de T-04, que ya está en Could), US-07 Cambiar contraseña propia, US-09 Corregir y reenviar solicitud rechazada	
E02 Barrios y Operadores	US-10 Alta de barrio, US-11 Modificar barrio, US-12 Activar/desactivar barrio, US-13 Crear Operador, US-15 Listar Operadores, US-45 Asignar	US-14 Reemplazar Operador, US-16 Modificar Operador, US-17 Cambiar contraseña del Operador	

	Operador a barrio		
E02 Empresas y Monotributistas	US-18 Listar Empresas y Monotributistas, US-20 Aprobar o rechazar	US-19 Filtrar por estado, US-21 Suspender y reactivar, US-22 Informe general	
E03 Trabajadores	US-23 Monotributista como su trabajador, US-24 Perfil propio del Monotributista, US-25 Registrar trabajador, US-26 Listar y modificar, US-27 Baja de trabajador, US-28 Cargar y renovar documentación	US-29 Filtrar por estado documental, US-30 Reporte documental, US-31 Historial propio, US-46 Historial del Monotributista	
E04 Operador	US-32 Gestionar Guardias, US-33 Listar aprobados, US-34 Habilitar/deshabilitar en el barrio, US-35 Filtrar por condición (marcada Must en el Trello), US-36 Historial del barrio		
E05 Garita	US-37 Buscar por DNI, US-38 Verificar y ver motivo, US-39 Registrar ingreso, US-40 Registrar egreso		
E06 Seguridad	US-41 Aislar Empresas, US-42 Aislar barrios, US-43 Restringir por estado del rol, US-44 Política de contraseñas		
Transversales	T-03 Aislamiento, T-05 Almacenamiento de archivos, T-06 Paginación y orden	T-04 Servicio de correo	

Donde no se llegue con lo secundario, se puede dejar una respuesta estática, por ejemplo US-22 y US-30 con datos fijos. La suspensión (US-21) queda como secundaria porque arrastra el egreso automático; si el profesor la considera parte del rol Administrador, se sube a S4. US-12 sí es core, pero su egreso automático al desactivar (criterio 5) puede llegar en S4.

Reparto por integrante

Propongo 5 bloques de 6 a 8 ítems core cada uno, agrupados para que cada persona dependa lo menos posible de otra. Cada persona hace backend, frontend y casos de prueba de sus US. Persona 1 es Juan, que ya tiene US-04 en curso; el resto lo eligen en la reunión.

Integrante	Bloque	Core	Secundarias	Pieza común que arma primero
Persona 1 (Juan)	Sesión y seguridad	US-01, 04, 05, 44, 41, 42, 43 (7)	US-06, 07, T-04	Aislamiento (T-03) dentro de US-42
Persona 2	Barrios, Operadores y Guardias	US-10, 11, 12, 13, 15, 45, 32 (7)	US-14, 16, 17	
Persona 3	Registro, solicitudes y habilitación	US-02, 03, 08, 18, 20, 33, 34, 35 (8)	US-09, 19, 21, 22	FileStorage (T-05) dentro de US-03
Persona 4	Trabajadores y documentos	US-23, 24, 25, 26, 27, 28 (6)	US-29, 30	
Persona 5	Garita e historial	módulo movimiento, US-37, 38, 39, 40, 36 (6)	US-31, 46	Módulo movimiento y función de PostgreSQL de la garita

T-03, T-05 y T-06 no se asignan aparte: las arma la primera US que las necesita, en common/, y las siguientes la reutilizan. Por eso conviene nombrar quién la arma, para que no aparezcan tres paginaciones distintas. Dependencias a cuidar: US-25 y US-28 usan el FileStorage de US-03; la paginación (T-06) ya está en common/paging y la usan todos los listados; US-23 y US-24 necesitan US-20. El módulo movimiento no tiene tarjeta en el Trello pero lo usan US-12, 21 y la garita.

Plan semana a semana

El orden sigue las dependencias: primero lo transversal y movimiento (que usan perfil, barrio y garita), después los datos que la garita necesita, y la garita completa en S3. Así el viernes 13/11 ya se puede mostrar el camino feliz entero y quedan dos semanas de margen.

Semana	Meta de la semana	Persona 1 (Juan)	Persona 2	Persona 3	Persona 4	Persona 5
S1: 24/10 al 30/10	Login andando y piezas comunes listas	US-04 Iniciar sesión (terminar) + US-44 Política de contraseñas	US-10 Alta de barrio + US-11 Modificar barrio	US-02 Crear cuenta + US-03 Solicitar perfil (arma FileStorage)	US-25 Registrar trabajador, sin archivos + US-26 Listar y modificar	Módulo movimiento + función verificar_ingreso en PostgreSQL
S2: 31/10 al 6/11	Admin y Empresa cargan sus datos	US-01 Activar cuenta del Admin + US-05 Cerrar sesión	US-13 Crear Operador + US-45 Asignar Operador	US-18 Listar solicitudes + US-20 Aprobar o rechazar	US-28 Documentación + foto en US-25	US-37 Buscar por DNI + US-38 Verificar y ver motivo

		sesión				
S3: 7/11 al 13/11	Camino feliz completo de punta a punta	US-42 Aislar barrios (arma aislamiento) + US-41 Aislar Empresas	US-12 Activar/desactivar barrio + US-32 Gestionar Guardias	US-33 Listar aprobados + US-34 Habilitar en el barrio	US-23 Monotributista como trabajador + US-27 Baja de trabajador	US-39 Registrar ingreso + US-40 Registrar egreso
S4: 14/11 al 20/11	Core cerrado y primeras secundarias	US-43 Restringir por estado + Dockerfiles y compose completo	US-15 Listar Operadores + secundaria US-14	US-08 Estado de mi solicitud + US-35 Filtro por condición	US-24 Perfil del Monotributista + secundaria US-29	US-36 Historial del barrio + test de la función con Testcontainers
S5: 21/11 al 27/11	Congelamiento y entrega	Todo el grupo: manual de instalación y anexo técnico, cada uno aporta la parte de su bloque. Cada uno ejecuta los casos de prueba de otro bloque y se prueba el camino feliz completo				

Si un ítem no se termina, pasa primero en la semana siguiente y se corre la secundaria de S4, nunca el core. Hasta que esté US-01, el Administrador se puede crear con un script de datos de prueba. El sábado 14/11 conviene hacer la demo interna del camino feliz para detectar huecos con tiempo.

Acuerdos para la semana 1

Antes de que arranquen las ramas de las US, conviene dejar listas estas cuatro cosas entre el sábado 24/10 y el lunes 26/10. Son chicas, pero si cada uno las resuelve por su lado, aparecen cinco versiones distintas.

Acuerdo	Qué se define	Estado
Docker para la infraestructura	docker-compose.yml con Postgres (y Mailpit cuando haga falta). Backend y	Hecho en el commit 2a54de1, falta pushear

	frontend corren local desde el IDE	main y dev
Seed base, opcional	Se elige en backend/.env: SPRING_PROFILES_ACTIVE=dev (sin datos) o dev,seed (con datos). El perfil seed le suma db/seed a Flyway. Para US-01 está la base vacía accesos_vacia (DB_NAME)	Hecho en el commit 2a54de1, falta pushear
Paginación común	PageResponse<T> en common/paging y su tipo en frontend/src/shared/types/page.ts. Tamaño por defecto 20, máximo 100	Hecho en el commit 2a54de1
Convenciones de DTO	record de Java, XxxRequest y XxxResponse en la carpeta dto de cada módulo, conversión con un método estático from() dentro del record, Bean Validation con mensajes en español, nunca devolver entidades	A confirmar en la reunión del 24/10
Formato de la planilla de testing	Columnas y forma de escribir cada caso, para que todos escriban igual	Resuelto: se usa el template que dio la cátedra

Seed base. Cubre el camino feliz, no todo el sistema:

- Un Administrador, un Operador y un Guardia con una contraseña conocida (hash bcrypt de, por ejemplo, Prueba123).
- Un barrio Activo con su Operador y otro Inactivo sin Operador.
- Una Empresa aprobada y habilitada en el barrio activo, un Monotributista aprobado, uno Pendiente y uno Rechazado con motivo.
- Trabajadores con documentación vigente, vencida y pendiente de carga, y un mismo DNI en dos vínculos.
- Un ingreso abierto, para probar el egreso y el error de ingreso activo.
- Las rutas de PDF y fotos ya están, pero sin archivo real: la ruta es relativa a STORAGE_PATH, no al repo. Los archivos de ejemplo se suman cuando esté el FileStorage de US-03.

Reglas para escribir seeds y migraciones, que ya están en el README:

- Cada US que necesite más casos agrega su propio archivo de seed fechado en su PR, sin editar los anteriores. Como hay ramas en paralelo, en dev está activado out-of-order para que Flyway acepte un script con fecha anterior al último aplicado. En prod no.
- Los IDs no se escriben a mano, porque las tablas los generan solas. Las filas relacionadas se insertan buscando por un dato único, por ejemplo INSERT … SELECT id FROM usuario WHERE email = 'operador@seed.test'.
- Una columna NOT NULL nueva siempre lleva DEFAULT o completa los datos en el

mismo script, porque las bases con seed ya tienen filas.

- Flyway no deshace un seed ya aplicado. Para probar US-01 se usa la base accesos_vacia, sin borrar la otra. Los tests con Testcontainers arrancan siempre con la base vacía.

Docker. En S4 se suman los Dockerfiles del backend y del frontend y el compose completo, y alguien lo levanta de cero en otra máquina. El manual de instalación depende de eso, así que no se deja para S5.

DTO, explicado

Un DTO (Data Transfer Object) es una clase simple con solo los campos que viajan entre el frontend y el backend en una petición o una respuesta. No tiene lógica, y no es lo mismo que la entidad.

La entidad representa la tabla tal cual. Por ejemplo, `Trabajador` tiene id, perfil, dni, foto_ruta, fecha_baja, creado_en y la lista de documentos. El DTO es lo que la pantalla necesita ver o mandar:

- `TrabajadorRequest` es lo que el frontend envía al dar de alta: nombre, apellido y dni.
- `TrabajadorResponse` es lo que el backend devuelve en el listado: id, nombre, apellido, dni, estado documental y si tiene foto.

```
public record TrabajadorRequest(
    @NotBlank String nombre,
    @NotBlank String apellido,
    @NotBlank String dni) {}

public record TrabajadorResponse(Long id, String nombre, String apellido,
                                 String dni, String estadoDocumental, boolean
tieneFoto) {
    public static TrabajadorResponse from(Trabajador t, String estadoDocumental)
{
        return new TrabajadorResponse(t.getId(), t.getNombre(), t.getApellido(),
                                      t.getDni(), estadoDocumental,
t.getFotoRuta() != null);
    }
}
```

El Controller recibe el Request, se lo pasa al Service, y devuelve el Response. La entidad nunca sale del backend. Hay tres motivos para no devolverla directamente:

1. **Filtra datos.** Devolver `Usuario` manda el password_hash al navegador. Devolver `Trabajador` puede exponer ids de otra Empresa, que es justo lo que prohíbe US-41.
2. **Rompe con Hibernate.** Las relaciones se cargan bajo demanda; al convertir a JSON aparecen errores o ciclos infinitos (Trabajador, Perfil, sus trabajadores, otra vez Perfil…).
3. **Ata el frontend a la base.** Si cambia una columna, cambia el JSON y se rompe la

pantalla, aunque nadie la haya tocado.

Qué acordar en común y qué pasa si no

Acuerdo	Qué pasa si no se acuerda	Gravedad
Nunca devolver entidades, siempre un Response	Se filtran contraseñas y datos de otros perfiles, y aparecen errores de Hibernate al serializar	Grave: es un problema de seguridad (RNF-06, US-41)
Listados paginados con `PageResponse<T>`	Cada listado devuelve otra forma (una lista, el Page de Spring, otra cosa) y la tabla genérica del frontend no sirve para todos	Medio
Fechas en ISO (2026-10-24) y estados con los mismos textos de la base (ACTIVO, PENDIENTE)	Cada pantalla tiene que traducir fechas y estados a su manera, y aparecen bugs del tipo «Vigente» contra «VIGENTE»	Medio
Validaciones con anotaciones en el Request (`@NotBlank` , `@Email`)	Si alguno valida a mano en el Service, sus errores salen en otro formato y el frontend los muestra distinto	Medio
DTO entre módulos acordados entre las dos personas	La garita espera un campo que trabajador no devuelve, y se descubre al integrar	Medio
`record` , nombres XxxRequest/XxxResponse, carpeta dto/ y método from()	Cada uno escribe con otro estilo (clases con Lombok, MapStruct, nombres distintos). Funciona igual, pero cuesta encontrar y leer código ajeno	Leve

En resumen: lo único grave es devolver entidades. Lo de fechas, estados y PageResponse genera bugs molestos al integrar. El estilo es prolijidad. Si en la reunión hay poco tiempo, acuerden por lo menos las tres primeras filas.

Los DTO compartidos que ya existen son `PageResponse<T>` y `FieldErrorItem` (formato de errores). `CurrentUser` se arma con US-04 y tiene que devolver los campos que ya espera el frontend en useCurrentUser.ts (email y roles). En el frontend, page empieza en 0 y la tabla de Ant Design en 1.

Documentación

Los casos de prueba los escribe cada desarrollador al terminar su US. El anexo técnico y el manual se hacen al final, entre todos, en S5. Para que esa semana alcance, cada uno anota sobre la marcha en un archivo compartido lo que el manual va a necesitar (variables de entorno, datos de prueba, decisiones que cambiaron), así en S5 se arma y no se reconstruye de memoria.

Documento	Quién	Cuándo	Qué hay que hacer
Casos de prueba (planilla de testing)	Quien desarrolló cada US	Al terminar cada US, antes de mergear	Un caso por criterio de aceptación como mínimo, más los de error (permisos, archivos inválidos, contraseña débil). Se usa el template que dio la cátedra
Anexo técnico	Todo el grupo	S5	Partir de decisiones_tecnicas_v2 y del modelo de T-01; cada uno completa la parte de sus módulos
Manual de instalación	Todo el grupo	S5	Docker Compose, variables de entorno, carpeta de archivos y entrega del token del Admin inicial (RF-01). Lo prueba alguien en una máquina limpia
Mockups	Ya hechos	S5	Actualizar solo si alguna pantalla cambió al implementarla

Definición de terminado y riesgos

Una US está terminada cuando cumple las cuatro condiciones:

- [ ] Backend y frontend integrados en la rama principal y andando con Docker Compose
- [ ] Cumple sus criterios de aceptación, probado por otra persona del equipo
- [ ] Sus casos de prueba están escritos en la planilla de testing
- [ ] Si cambia el esquema, trae su migración de Flyway nueva junto con la entidad

Riesgo	Señal temprana	Plan B
La función de la garita se complica	Al viernes 6/11 US-38 no devuelve habilitado y motivo	Verificar en el Service con JPA y pasar a función en diciembre
Archivos bloquean registro y trabajadores	T-05 sin terminar el miércoles 28/10	Guardar la ruta con un archivo de prueba y validar tipo y tamaño después
El login atrasa a todos	US-04 sin JWT funcionando el	Trabajar con un usuario y rol

El login atrasa a todos	US-04 sin JWT funcionando el 30/10	Trabajar con un usuario y rol fijos en desarrollo y conectar el JWT apenas esté
Casos de prueba quedan para el final	Planilla vacía en la reunión del 7/11	En la reunión semanal, una US sin casos no se cuenta como hecha

Ramas, PRs y cómo evitar stubs

Recomiendo no dejar PRs abiertos esperando a otra US: mergear a dev cuando la US funciona contra la base real, y cubrir lo que falta con datos de prueba en la base, no con código falso. Un PR abierto una o dos semanas choca con las migraciones de Flyway y con common/, y al final se integra todo junto, que es justo lo que se quiere evitar.

1. Una rama feature por 1 o 2 US, que sale de dev y vuelve a dev en menos de una semana. Traer dev a la rama todos los días.
2. Sin stubs en src/main: los mocks solo en los tests. Si una US necesita datos que otra todavía no carga (por ejemplo, la garita necesita trabajadores antes de que exista la pantalla de US-25), se insertan con un script de datos de prueba.
3. Datos de prueba con Flyway en una carpeta aparte (por ejemplo db/seed) que solo se carga con el perfil seed. Así son tablas y datos reales, y en prod no aparecen.
4. Contrato primero: antes de empezar dos US que se tocan, acordar el endpoint y el DTO en Swagger, o la firma del Service. El que consume llama al Service real del otro módulo apenas está en dev.
5. Separar «mergeada» de «terminada». Mergeada es que pasa sus tests y funciona con datos de prueba. Terminada es que sus criterios de aceptación se cumplen con los datos que cargan las otras US y tiene sus casos de prueba escritos. Solo la terminada cuenta para el 70%.
6. El plan ya ordena las US para que quien produce los datos vaya una semana antes de quien los consume, así que los casos que dependan de datos de prueba deberían ser pocos.

Ajustes al Trello

Revisé las 46 US y 6 tareas del tablero. La numeración del plan ya sigue el Trello; esto es lo que conviene corregir o agregar.

Qué	Dónde	Propuesta
Solo US-01 tiene el ítem «Testing: escribir los casos de prueba en la planilla»	Checklists de todas las demás US	Agregarlo a todas, ya que es parte de la definición de terminado
No hay tarjetas para la	Backlog	Crear D-01 Manual de instalación,

documentación		D-02 Anexo técnico y D-03 Formato de la planilla de testing, con responsable y semana
No hay tarjeta para el módulo movimiento	Backlog	Crear una tarea transversal (por ejemplo T-07), que la usan US-12, 21 y 39 a 40
No hay tarjeta para Docker Compose	Backlog	Sumarla a T-02 o crear una tarea, porque el manual de instalación depende de eso
Solo US-35 tiene prioridad (Must)	Todas las US	Etiquetar Must o Could según la tabla de cero de este doc, para